Result Processing System

- Backend API for student results, GPA/CGPA, analytics and academic insights

1. Problem

- Manual result processing can be slow and error-prone.
- Institutions need consistent GPA/CGPA calculations.
- Academic staff need ranking, course analysis and performance trends.
- Students at academic risk should be identifiable from verified results.

2. Project Approach

- FastAPI REST backend with SQLite persistence.
- Pydantic schemas validate API data.
- Service modules separate business/database operations from routes.
- Pandas performs academic and performance analytics.

3. System Architecture

- Client / Swagger UI → FastAPI routes → schemas/auth → services + analytics → SQLite.
- Verified result records feed academic calculations, analytics and insight generation.

4. Data Handling

- Users, students, courses and results are stored in SQLite.
- Results contain student/course links, score, semester and session.
- Foreign keys, uniqueness constraints, validation checks and indexes support integrity.

5. Core Result Logic

- A: 70–100 = 5.0 | B: 60–69 = 4.0 | C: 50–59 = 3.0
- D: 45–49 = 2.0 | E: 40–44 = 1.0 | F: 0–39 = 0.0
- Quality Point = Grade Point × Credit Unit
- GPA = Σ Quality Points / Σ Credit Units

6. API Features

- Student, course and result CRUD operations.
- GPA, CGPA and academic history endpoints.
- Registration and JWT login.
- Admin dashboard protected by role-based access control.

7. Analytics

- Student statistics and GPA ranking.
- Class average, pass/failure rates and course statistics.
- Strongest/weakest courses and grade distribution.
- At-risk students and performance trends.

8. Academic Insights

- AIInsightService consumes verified analytics rather than calculating academic results.
- Produces human-readable explanations, reasons and deterministic recommendations.

9. Security & Error Handling

- bcrypt password hashing and JWT access tokens.
- Roles: admin, lecturer and student.
- Custom exceptions for not-found, duplicate, database and validation errors.
- Controlled unexpected-error responses.

10. Testing

- 13 test modules and 106 test functions are included in the project dump.
- Tests cover academic logic, analytics, AI insights, API behavior, authentication, database behavior, errors and performance.
- Tests use an isolated temporary SQLite database.

11. Results / Output

- Example: 33 total quality points / 8 credit units = GPA 4.13.
- The API returns JSON for GPA/CGPA, analytics and performance insights.

12. Demonstration Flow

- Start Uvicorn → open /docs → authenticate → create data → add results → calculate GPA/CGPA.
- Then demonstrate analytics, at-risk detection, insight generation and pytest -v.

13. Conclusion

- End-to-end backend workflow for academic result processing.
- Combines CRUD, validation, persistence, calculations, analytics, security and testing.
- Final submission should include the GitHub URL, screenshots and presentation link.